

THE DEFINITIVE ARCHITECTURE GUIDE: 2D PONG IN MODERN OPENGL

A Comprehensive Deep Dive into Core Profile Graphics, Buffer Objects (VBO, VAO, EBO), Custom Engine Abstractions, Shaders, Bitmapped Typography, Real-time Physics, and Systems Engineering.

ABOUT THIS MANUAL

What This Mini-Book Covers:

- **The Modern Graphics Pipeline:** How CPU and GPU cooperate in OpenGL 4.6 Core Profile.
- **The Buffer Trinity (VBO, VAO, EBO):** Exact memory layouts, stride calculations, attribute binding mechanics, and index caching.
- **Custom Engine Architecture:** Line-by-line breakdown of VAO, VBO, EBO, Shader, and Texture classes.
- **The Unit Quad Paradigm:** How a single 4-vertex geometry renders all paddles, ball, net, and text via shader uniforms.
- **5x3 Retro Bitmap Font Engine:** Bitwise bitmask extraction decoding retro characters without external font libraries.
- **Dynamic Game Physics:** Trigonometric angle reflection, compounding velocity acceleration, and anti-tunneling math.
- **Game Loop & Concurrency:** GLFW windowing, Delta Time clamping, state debouncing, and asynchronous audio.

Project: Ping_Pong_2D | **Language:** C++20 / GLSL 460 | **Libraries:** GLFW 3, GLAD, stb_image
Architecture Target: Modern OpenGL Core Profile (v4.6) | **OS:** Windows 64-bit MinGW-w64

Chapter 1: The Modern OpenGL Rendering Philosophy

1.1 The Paradigm Shift: Immediate Mode vs Core Profile

In vintage OpenGL (pre-3.0, known as 'Immediate Mode'), developers issued rendering commands using functions like `glBegin(GL_TRIANGLES)`, `glVertex3f(...)`, and `glEnd()`. In this old paradigm, vertex data was transmitted from the CPU to the GPU on every single draw call, creating a devastating bottleneck over the PCIe bus. Furthermore, lighting and projection relied on fixed, unmodifiable hardware matrices.

Modern OpenGL (specifically Core Profile 3.3 up through 4.6, as used in this Pong project) completely removed all fixed-function code. In Core Profile:

- 1. **Zero Immediate Calls:** You cannot draw anything without providing your own custom GLSL shaders.
- 2. **GPU-Resident Data:** All geometry data (vertex positions, texture coordinates) must be pre-uploaded into the GPU's high-speed video RAM (VRAM) ahead of time.
- 3. **State Separation:** The memory holding the raw floats (VBO) is decoupled from the memory layout blueprint (VAO).

1.2 The Programmable Graphics Pipeline Stages

Every single frame rendered in Pong passes through a deterministic series of hardware pipeline stages:

Pipeline Stage	Execution Unit	Pong 2D Role / Implementation
Vertex Fetch	Hardware	Reads raw vertex bytes from GPU VRAM governed by the active VAO.
Vertex Shader	Programmable (GPU)	Applies scale, position offsets, and transforms NDC coordinates (<code>pong.vert</code> / <code>default.vert</code>).
Primitive Assembly	Hardware	Connects transformed vertices into triangles using indices from the EBO.
Rasterization	Hardware	Interpolates triangles across pixels on screen; determines which fragments are covered.
Fragment Shader	Programmable (GPU)	Calculates the final RGBA color of each pixel (solid white/gold or textured CRT noise).
Framebuffer Tests	Hardware	Performs blending and writes final pixels to the double-buffered swap chain.

1.3 Normalized Device Coordinates (NDC)

Regardless of whether the game window is 800x800, 1920x1080, or 4K, modern OpenGL operates exclusively inside a normalized coordinate cube where both the X and Y axes range strictly from **-1.0 to +1.0**.

- **Center of Screen:** (0.0, 0.0)
- **Top-Left Corner:** (-1.0, +1.0) | **Bottom-Right Corner:** (+1.0, -1.0)

In our Pong engine, the playfield is bounded between $Y = +0.92$ (top border) and $Y = -0.92$ (bottom border), leaving room for top scores and bottom keyboard tips.

Chapter 2: The Core Trinity — VAO, VBO, and EBO

Understanding how **VBO**, **VAO**, and **EBO** collaborate is the single most critical concept in modern OpenGL programming. Beginners often confuse them because their names sound similar, but they perform three radically distinct functions in GPU memory.

Buffer / Object	Full OpenGL Name	Physical Location	Core Responsibility
VBO	Vertex Buffer Object	GPU Video Memory (VRAM)	Stores raw, unformatted binary arrays of vertex floats.
VAO	Vertex Array Object	GPU State Container	Stores the 'blueprint': memory stride, offsets, and attribute pointers.
EBO (IBO)	Element Buffer Object	GPU Video Memory (VRAM)	Stores index arrays telling the GPU which vertices form triangles.

2.1 VBO (Vertex Buffer Object): The Raw Data Store

A VBO is simply a chunk of dedicated memory allocated inside the graphics card. It has no idea what its data represents—it is merely an array of contiguous bytes. In our Pong game, we have two primary VBO data structures:

```
// 1. Unit Quad VBO (Used for paddles, ball, net, text)
GLfloat quadVertices[] = {
    0.0f, 0.0f, // Vertex 0: Bottom-Left (X, Y)
    1.0f, 0.0f, // Vertex 1: Bottom-Right (X, Y)
    1.0f, 1.0f, // Vertex 2: Top-Right (X, Y)
    0.0f, 1.0f // Vertex 3: Top-Left (X, Y)
}; // Total: 8 floats = 32 bytes

// 2. Fullscreen Background Quad VBO (Used for CRT texture)
GLfloat bgVertices[] = {
    // Position (X, Y) // UV Texture Coords (U, V)
    -1.0f, -1.0f,      0.0f, 0.0f, // Vertex 0: Bottom-Left
    1.0f, -1.0f,      1.0f, 0.0f, // Vertex 1: Bottom-Right
    1.0f, 1.0f,       1.0f, 1.0f, // Vertex 2: Top-Right
    -1.0f, 1.0f,      0.0f, 1.0f // Vertex 3: Top-Left
}; // Total: 16 floats = 64 bytes
```

2.2 EBO (Element Buffer Object): Eliminating Redundant Vertices

A rectangle is formed by two adjacent triangles. If you draw two triangles without an EBO, you must provide **6 vertices** (3 per triangle). That means 2 vertices must be duplicated, wasting GPU memory and processing bandwidth.

The **EBO (Index Buffer)** solves this completely: we upload only **4 unique vertices** (0, 1, 2, 3), and then upload an array of indices that tell the GPU how to connect them into two triangles:

```
GLuint indices[] = {
    0, 1, 2, // Triangle 1: Bottom-Left -> Bottom-Right -> Top-Right
    2, 3, 0 // Triangle 2: Top-Right -> Top-Left -> Bottom-Left
};
```

2.3 VAO (Vertex Array Object): The Blueprint State Container

Because a VBO is just a raw bucket of floats, the GPU cannot render it unless you tell it: *'Where does each vertex start? How many floats are in a position? What is the byte gap between vertices?'*

In early OpenGL, you had to call `glVertexAttribPointer` before every single draw call. The **VAO** was invented to store all of those configuration pointers inside an object. Once a VAO is configured, drawing that geometry in the render loop requires only **one line of code**: `VAO.Bind()`!

CRITICAL ARCHITECTURAL GOTCHA

Critical OpenGL Rule — What a VAO Records:

1. Every call to `glVertexAttribPointer` and `glEnableVertexAttribArray`.
2. The specific VBO that was bound to `GL_ARRAY_BUFFER` when `glVertexAttribPointer` was called.
3. The active **EBO** bound to `GL_ELEMENT_ARRAY_BUFFER` while the VAO is bound!

Golden Warning: Never call `glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0)` before unbinding your VAO, or you will unbind the EBO from the VAO itself!

2.4 Step-by-Step State Flow: How VAO, VBO, & EBO Link Together

Here is the precise sequence of operations executed during engine initialization:

```
// STEP 1: Generate and Bind the VAO
// All configuration calls after this point are recorded into this VAO.
VAO quadVAO;
quadVAO.Bind(); // glBindVertexArray(ID)

// STEP 2: Generate VBO and Upload Vertex Data to GPU VRAM
VBO quadVBO(quadVertices, sizeof(quadVertices)); // glGenBuffers, glBindBuffer, glBufferData

// STEP 3: Generate EBO and Upload Index Data to GPU VRAM
// Because quadVAO is active, the EBO is automatically registered inside quadVAO!
EBO quadEBO(quadIndices, sizeof(quadIndices)); // glGenBuffers, glBindBuffer, glBufferData

// STEP 4: Define the Memory Layout Blueprint
// Tell the VAO: layout=0, 2 components (X, Y), floats, stride=2*sizeof(float), offset=0
quadVAO.LinkAttrib(quadVBO, 0, 2, GL_FLOAT, 2 * sizeof(float), (void*)0);

// STEP 5: Unbind to prevent accidental state modification
quadVAO.Unbind(); // glBindVertexArray(0)
quadVBO.Unbind(); // glBindBuffer(GL_ARRAY_BUFFER, 0)
quadEBO.Unbind(); // glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0)

// =====
// IN THE GAME LOOP (Every Frame):
// =====
pongShader.Activate();
quadVAO.Bind(); // Instantly restores VBO attributes AND the bound EBO!
glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0); // Draws 2 triangles
quadVAO.Unbind();
```

2.5 Stride and Offset Mathematics in Vertex Attributes

The call to `glVertexAttribPointer(layout, size, type, normalized, stride, pointer)` contains two crucial arguments that frequently confuse graphics programmers:

Attribute Parameter	Meaning in Pong 2D	Quad VBO Value	Background VBO Value
layout (index)	Matches layout(location = X) in shader	0 (aPos)	0 (aPos) & 1 (aTex)
numComponents	Number of components per attribute	2 (X, Y)	2 for Pos, 2 for UV
type	Data type of each component	GL_FLOAT (4 bytes)	GL_FLOAT (4 bytes)
stride	Byte gap between successive vertices	2 * 4 = 8 bytes	4 * 4 = 16 bytes
offset (pointer)	Byte start offset of attribute in vertex	(void*)0	(void*)0 (Pos), (void*)8 (UV)

Chapter 3: Custom Engine Abstraction Classes

To prevent OpenGL code from turning into an unmaintainable tangle of raw integer IDs and boilerplate calls, this project wraps OpenGL primitives into clean, object-oriented C++ classes.

3.1 The VBO Class (VBO.h / VBO.cpp)

The VBO class encapsulates the creation, binding, and deletion of a GPU vertex buffer:

```
VBO::VBO(GLfloat* vertices, GLsizeiptr size) {
    glGenBuffers(1, &ID); // Generate unique GPU buffer ID
    glBindBuffer(GL_ARRAY_BUFFER, ID); // Bind target
    glBufferData(GL_ARRAY_BUFFER, size, vertices, GL_STATIC_DRAW); // Copy data from CPU RAM to GPU
    VRAM
}

void VBO::Bind() { glBindBuffer(GL_ARRAY_BUFFER, ID); }
void VBO::Unbind() { glBindBuffer(GL_ARRAY_BUFFER, 0); }
void VBO::Delete() { glDeleteBuffers(1, &ID); }
```

3.2 The VAO Class (VAO.h / VAO.cpp)

The VAO class acts as the conductor, providing the LinkAttrib method that bridges a VBO to a specific shader attribute location:

```
void VAO::LinkAttrib(VBO& VBO, GLuint layout, GLuint numComponents, GLenum type, GLsizeiptr stride,
void* offset) {
    VBO.Bind();
    glVertexAttribPointer(layout, numComponents, type, GL_FALSE, stride, offset);
    glEnableVertexAttribArray(layout);
    VBO.Unbind();
}
```

3.3 The Shader Class (shaderClass.h / ShaderClass.cpp)

The Shader class handles reading GLSL text files from disk, compiling the Vertex and Fragment shaders independently on the GPU, linking them into a unified GPU program, and inspecting the compilation status:

```
// Inside Shader::compileErrors:
glGetShaderiv(shader, GL_COMPILE_STATUS, &hasCompiled);
if (hasCompiled == GL_FALSE) {
    glGetShaderInfoLog(shader, 1024, NULL, infoLog);
    std::cout << "SHADER_COMPILATION_ERROR for: " << type << "\n" << infoLog << "\n";
}
```

3.4 The Texture Class (Texture.h / Texture.cpp)

The Texture class leverages Sean Barrett's industry-standard stb_image library to load raw PNG image bytes from disk, flip image coordinates to match OpenGL's bottom-left origin, and configure texture sampling parameters:

```
stbi_set_flip_vertically_on_load(true);
unsigned char* bytes = stbi_load(image, &widthImg, &heightImg, &numColCh, 0);
glGenTextures(1, &ID);
glActiveTexture(slot); // e.g., GL_TEXTURE0
glBindTexture(texType, ID);
glTexParameteri(texType, GL_TEXTURE_MIN_FILTER, GL_NEAREST_MIPMAP_LINEAR);
glTexParameteri(texType, GL_TEXTURE_MAG_FILTER, GL_NEAREST); // Crisp pixel art
glTexImage2D(texType, 0, GL_RGBA, widthImg, heightImg, 0, format, pixelType, bytes);
glGenerateMipmap(texType);
stbi_image_free(bytes);
```

Chapter 4: Shaders & The Unit Quad Architecture

A common beginner mistake in 2D game development is re-uploading new vertex coordinates to the GPU whenever a paddle moves or the ball bounces. In this project, we employ an elite technique: **The Unit Quad Architecture**.

4.1 The Unit Quad Transform Mathematics

We upload exactly **one** square quad with local coordinates $[0.0, 1.0] \times [0.0, 1.0]$ to the GPU at startup. Inside `pong.vert`, the vertex shader dynamically stretches and moves this unit quad using two uniform vectors: `scale` and `offset`:

```
#version 460 core
layout (location = 0) in vec2 aPos; // Raw unit quad: (0,0), (1,0), (1,1), (0,1)

uniform vec2 offset; // Position on screen (NDC coordinates)
uniform vec2 scale; // Width and Height in NDC units

void main() {
    // Formula: TransformedPos = (UnitPos * Scale) + Offset
    gl_Position = vec4(aPos.x * scale.x + offset.x, aPos.y * scale.y + offset.y, 0.0, 1.0);
}
```

4.2 How Everything is Drawn with One Single Buffer

Because of this mathematical transformation in the shader, the CPU never has to modify GPU vertex memory! Look at how various game elements are drawn by simply changing uniforms:

Game Element	Scale Uniform (Width, Height)	Offset Uniform (X, Y)	Color Uniform (R, G, B)
Top Border	scale = (2.0f, 0.02f)	offset = (-1.0f, +0.92f)	White (1.0, 1.0, 1.0)
Net Dash	scale = (0.012f, 0.045f)	offset = (-0.006f, y_pos)	White (1.0, 1.0, 1.0)
Player 1 Paddle	scale = (0.032f, 0.25f)	offset = (-0.92f, p1_y - 0.125f)	White (1.0, 1.0, 1.0)
Square Ball	scale = (0.028f, 0.028f)	offset = (ball_x - 0.014f, ball_y - 0.014f)	White (1.0, 1.0, 1.0)
Font Pixel	scale = (pixelW*0.95, pixelH*0.95)	offset = (startX, py)	Ice Cyan (0.2, 0.9, 1.0)

4.3 The Fragment Shaders: Solid Color vs CRT Noise

- **pong.frag**: Uses a simple uniform `vec3 color`; to paint pixels solid white, gold, or ice-cyan.
- **default.frag**: Uses uniform `sampler2D tex0`; and texture coordinates from `default.vert` to sample `background.png`, giving the screen its authentic 1970s CRT grain look.

Chapter 5: The 5x3 Retro Bitmap Font Engine

Most OpenGL games rely on heavy external libraries like FreeType or font texture atlases to display text. In this project, we implemented a custom **Bitwise Bitmap Font Generator** right inside C++ without any third-party dependencies.

5.1 Bitwise Glyph Representation

Every character in our font is 5 rows high by 3 columns wide. A 3-column row requires only **3 bits** of data! Therefore, an entire 5x3 glyph is represented as an array of 5 unsigned 8-bit integers (uint8_t):

- Bit 2 (0b100) = Left column
- Bit 1 (0b010) = Middle column
- Bit 0 (0b001) = Right column

```
// Example: The digit '0' (5 rows high, 3 columns wide)
static const uint8_t FONT_0[5] = {
    0b111, // Row 0: ■ ■ ■ (Top bar)
    0b101, // Row 1: ■   ■ (Left & Right walls)
    0b101, // Row 2: ■   ■ (Left & Right walls)
    0b101, // Row 3: ■   ■ (Left & Right walls)
    0b111  // Row 4: ■ ■ ■ (Bottom bar)
};

// Example: The letter 'P' (Used in 'PAUSED')
static const uint8_t FONT_P[5] = {
    0b111, // Row 0: ■ ■ ■
    0b101, // Row 1: ■   ■
    0b111, // Row 2: ■ ■ ■
    0b100, // Row 3: ■
    0b100  // Row 4: ■
};
```

5.2 Decoding Bits into OpenGL Quads

Inside RenderChar(...), the engine loops over each row from 0 to 4, and uses the bitwise AND operator (&) to test whether a pixel should be drawn:

```
void RenderChar(char c, float startX, float startY, float pixelW, float pixelH, GLuint offsetLoc, GL
uint scaleLoc) {
    const uint8_t* rows = GetCharBitmap(c);
    glUniform2f(scaleLoc, pixelW * 0.95f, pixelH * 0.95f);

    for (int r = 0; r < 5; r++) {
        uint8_t rowVal = rows[r];
        float py = startY + (4 - r) * pixelH; // Invert row so row 0 is at top

        if (rowVal & 0b100) { // Test Left Column
            glUniform2f(offsetLoc, startX, py);
            glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
        }
        if (rowVal & 0b010) { // Test Middle Column
            glUniform2f(offsetLoc, startX + pixelW, py);
            glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
        }
        if (rowVal & 0b001) { // Test Right Column
            glUniform2f(offsetLoc, startX + 2.0f * pixelW, py);
            glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);
        }
    }
}
```

By setting pixelW and pixelH, the exact same function draws the massive scoreboard numbers (0.022x0.026), the medium win counters (0.010x0.012), and the small controls help text (0.007x0.009)!

Chapter 6: Game Physics, Collisions, and Math

Pong's classic feel depends entirely on its ball bounce deflection mathematics and progressive rally acceleration.

6.1 Axis-Aligned Bounding Box (AABB) Collision Detection

To check if the ball hits a paddle, we perform an AABB overlap test in 2D space. For Player 1 (Left paddle):

- Ball X bounds: [ball_x - halfBall, ball_x + halfBall]
- Paddle X bounds: [p1_x, p1_x + PADDLE_WIDTH]
- Ball Y bounds: [ball_y - halfBall, ball_y + halfBall]
- Paddle Y bounds: [p1_y - halfPaddleH, p1_y + halfPaddleH]

Additionally, we verify that ball_vx < 0.0f so that collision triggers only when the ball is traveling *toward* the paddle.

6.2 Dynamic Trigonometric Angle Deflection

If a paddle behaved like a flat wall, the ball would simply bounce off at the same angle it entered, making the game boring. In our engine, the reflection angle depends entirely on **where on the paddle** the ball strikes:

```
// 1. Calculate relative hit position normalized from -1.0 (bottom edge) to +1.0 (top edge)
float relativeHit = (ball_y - p1_y) / halfPaddleH;
relativeHit = std::clamp(relativeHit, -1.0f, 1.0f);

// 2. Map relative hit to a deflection angle up to +/- 55 degrees
float bounceAngle = relativeHit * (55.0f * 3.14159265f / 180.0f);

// 3. Accelerate ball velocity (1.05x speed per paddle strike)
currentBallSpeed = std::min(currentBallSpeed * BALL_SPEED_INCREMENT, BALL_MAX_SPEED);

// 4. Resolve new velocity components using trigonometry
ball_vx = currentBallSpeed * cos(bounceAngle); // Positive X (toward right)
ball_vy = currentBallSpeed * sin(bounceAngle);
```

6.3 Progressive Rally Acceleration & Wall Bounces

- **Speed Multiplier:** Every time a paddle successfully hits the ball, currentBallSpeed multiplies by 1.05f (5% faster).

This creates tense, fast-paced rallies that max out at BALL_MAX_SPEED = 2.4f.

- **Wall Inversion:** When striking the top (Y = +0.92) or bottom (Y = -0.92) boundary, the Y velocity component is inverted (ball_vy = -abs(ball_vy)), and the ball's position is clamped to prevent it from escaping the arena.

Chapter 7: Game Loop, Timing & Systems Architecture

7.1 Delta Time & The Anti-Tunneling Clamp

Games must run at the exact same physical speed regardless of whether a player's monitor runs at 60Hz, 144Hz, or 240Hz. We calculate $\text{deltaTime} = \text{currentFrameTime} - \text{lastFrameTime}$ on every tick.

However, if a user drags the window or a lag spike occurs, deltaTime could jump to 0.5s, which would cause the ball to 'tunnel' right through the paddle! We clamp delta time to protect against tunneling:

```
float currentFrameTime = (float)glfwGetTime();
float deltaTime = currentFrameTime - lastFrameTime;
lastFrameTime = currentFrameTime;
if (deltaTime > 0.05f) deltaTime = 0.05f; // Clamped to 50ms max step
```

7.2 Edge-Triggered Input & The 'P' Freeze Key

In GLFW, `glfwGetKey(window, GLFW_KEY_P)` returns `GLFW_PRESS` on every single frame that the key is held down. If you simply wrote `if (pressed) isPaused = !isPaused;`, the game would toggle pause 144 times a second!

We implemented **Edge-Triggered Debouncing**:

```
bool pKeyPressed = (glfwGetKey(window, GLFW_KEY_P) == GLFW_PRESS);
if (pKeyPressed && !pKeyWasPressed) {
    isPaused = !isPaused; // Only fires once on the rising edge!
}
pKeyWasPressed = pKeyPressed;

if (!matchOver && !isPaused) {
    // Only update paddles and ball physics when NOT paused!
}
```

7.3 Asynchronous Audio Synthesis via Windows Beep

The Win32 API provides `Beep(DWORD dwFreq, DWORD dwDuration)`. However, `Beep()` is a **synchronous blocking call**—if you call `Beep(540, 100)` on the main thread, the entire game will freeze for 100 milliseconds!

We solved this by spawning detached C++ worker threads:

```
void PlaySoundAsync(int freq, int durationMs) {
#ifdef _WIN32
    std::thread([]() {
        Beep(freq, durationMs); // Executes on background thread
    }).detach(); // Thread terminates automatically
#endif
}
```

Chapter 8: Compilation & Toolchain Breakdown

The compilation command inside build.ps1 turns raw C++ code into a high-performance Windows binary:

Compiler Flag	Technical Purpose in Pong 2D
-std=c++20	Enables modern C++ features: lambdas, auto types, std::clamp, and standard concurrency.
-mwindows	Targets the Windows GUI subsystem; suppresses the black command prompt window.
-static	Statically links C/C++ runtimes into the .exe so it runs on PCs without MinGW installed.
-static-libgcc	Embeds the GCC runtime library directly inside the binary.
-static-libstdc++	Embeds the C++ Standard Template Library directly inside the binary.
-Isource/engine	Tells the preprocessor where to search for header files (#include).
-lglfw3	Links the static GLFW library (window management, input, OpenGL context).
-lopengl32	Links Microsoft Windows' core OpenGL driver interface.
-lgdi32	Links the Windows Graphics Device Interface (required for GLFW window creation).

ARCHITECTURAL EXCELLENCE

Summary & Architectural Checklist:

- **VBO:** Holds the raw unit quad vertices on the GPU.
- **VAO:** Remembers vertex attribute formatting and links to the VBO & EBO.
- **EBO:** Instructs the GPU to draw two triangles using 4 indices.
- **Shaders:** Scales and offsets the unit quad dynamically per draw call.
- **Bitmap Font:** Draws text pixel-by-pixel using bitwise masks and the unit quad.
- **Threads:** Plays retro beeps asynchronously without stalling the rendering loop.